

Resolução dos Exercícios

Capítulo: Macros e o pré-processador

1. Exercício de Exame

Suponha as seguintes constantes simbólicas declaradas:

```
#define TAXA_ICMS1 5
```

```
#define TAXA_ICMS2 17
```

- 1.1 Defina a macro `Val_ICMS(Salario)` que deverá devolver o valor do ICMS de um salário, sabendo que a `TAXA_ICMS1` se o salário é menor que 10000, ou `TAXA_ICMS2`, caso contrário.

[Ver código-fonte \(exe01-1.c\)](#)

- 1.2 Qual a expansão de código que o pré-processador faria do seguinte trecho

```
main()
{
    int x, y = 2;
    printf("\nSalário=%f", Val_ICMS(x+y));
}
```

```
main()
{
    int x, y = 2;
    printf("\nSalário=%f",
        (((x+y) < 10000) ?
        ((x+y) * TAXA_ICMS1 / 100)) :
        ((x+y * TAXA_ICMS2 / 100)));
}
```

```
}
```

2. Exercício de Exame

Indique qual a ordem pela qual as operações são realizadas no desenvolvimento de uma aplicação

- "Linkagem" **4**
- Edição de código **1**
- Compilação **3**
- Execução da aplicação **5**
- Pré-processamento **2**

3. Exercício de Exame

a) Escreva a macro ZAP(x, v1, v2) que, caso $x \leq 0$, devolve $x*v1$, se não devolve $x*(-v2)$.

```
#define ZAP(x, v1, v2) (((x) <= 0) ?  
                        ((x) * (v1)) :  
                        ((x) * (- (v2))))
```

b) Qual a expansão que o pré-processador faria do seguinte código:

```
#define MAX 43  
#define MIN -1  
  
main()  
{  
    int i, j = (int) 'a';  
    i = ZAP(i+j, MAX-1, MIN-1);  
}
```

```
#define MAX 43
#define MIN -1

main()
{
    int i, j = (int) 'a';
    i = (((i+j) <= 0) ?
        (i+j) * (43 - 1) :
        (i+j) * (-(-1 - 1)));
}
```

4. Exercício de Exame

- **4.1 Por que é que os parâmetros existentes nas macros não têm tipos de dados?**

Como as macros são instruções para o pré-processador e não instruções da linguagem C, o pré-processador não conhece os tipos existentes na linguagem. Por isso, não são definidos os tipos de dados que serão trabalhados na macro.

- **4.2 Defina a macro `toupper(ch)` que deverá converter o valor de `ch` para maiúsculas**

```
#define toupper(ch) (((ch) >= 'a' && (ch) <= 'z') ?
    (ch) + 'A' - 'a' : (ch))
```

- **4.3 Qual a expansão que seria produzida pelo pré-processador a partir do seguinte trecho, utilizando a macro definida no item anterior?**

```
#define FIRST 'A'
#define LAST 'Z'

main()
```

```
{
    int ch=65; /* ASCII do 'A' */
    printf("%c %c", toupper(FIRST+2),toupper(LAST+'a'-ch));
}
```

```
#define FIRST 'A'
#define LAST 'Z'

main()
{
    int ch=65; /* ASCII do 'A' */
    printf("%c %c", (((('A'+2) >= 'a' && ('A'+2) <= 'z')
    ('A'+2) + 'A' - 'a' :
    ('A'+2)),
    (((('Z'+ 'a' -ch) >= 'a' && ('Z'+ 'a' -ch) <= 'z') ?
    (('Z'+ 'a' -ch) + 'A' - 'a') :
    ('Z'+ 'a' -ch))));
}
```

- 4.4 Qual a saída que seria gerada pela execução desse programa?

C Z